Accelerating Deep Learning Algorithms on Intel Xeon Phi

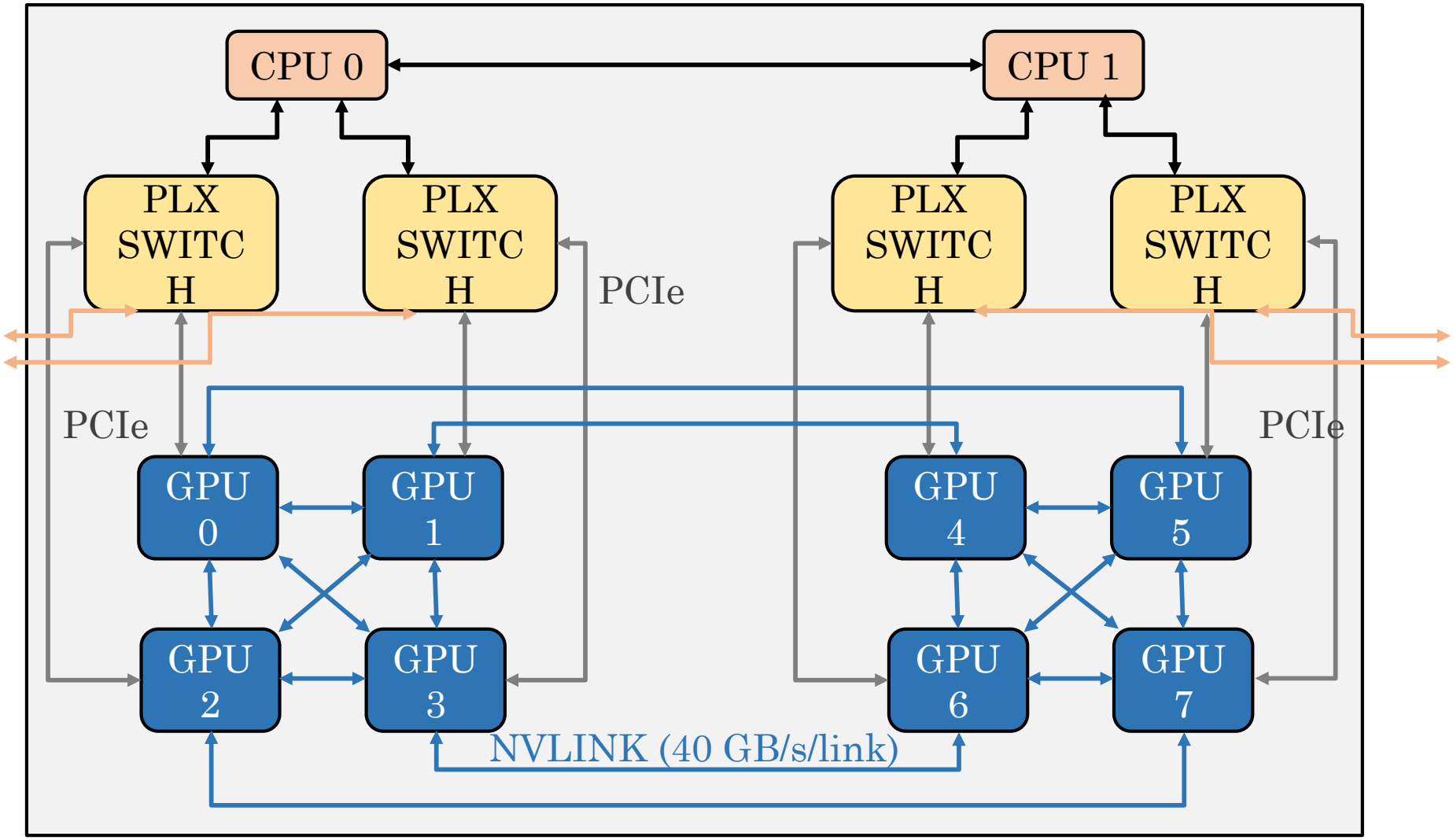
Sparsh Mittal

© Sparsh Mittal

Comparison of KNL with NVIDIA DGX-1

Gawande et al., “Scaling Deep Learning Workloads: NVIDIA DGX-1/Pascal and Intel Knights Landing”, IPDPSW, 2017

Architecture of NVIDIA DGX-1



Overview

- They compare NVIDIA DGX-1 (8 Pascal P100 GPUs with NVLink interconnect) against a cluster with Intel KNL and Omni-Path interconnect
- They evaluate vendor-provided models of four CNNs, viz., CifarNet, AlexNet, CaffeNet and GoogleNet.
  - CaffeNet is just single-GPU version of AlexNet, so results of CaffeNet and AlexNet are identical
- They use Caffe frameworks provided by respective vendors optimized for their systems.
- They ensure that testing error of CNNs after a large number of iterations was almost equal for two systems and was also equal to the sequential execution on Berkeley Caffe.

© Sparsh Mittal

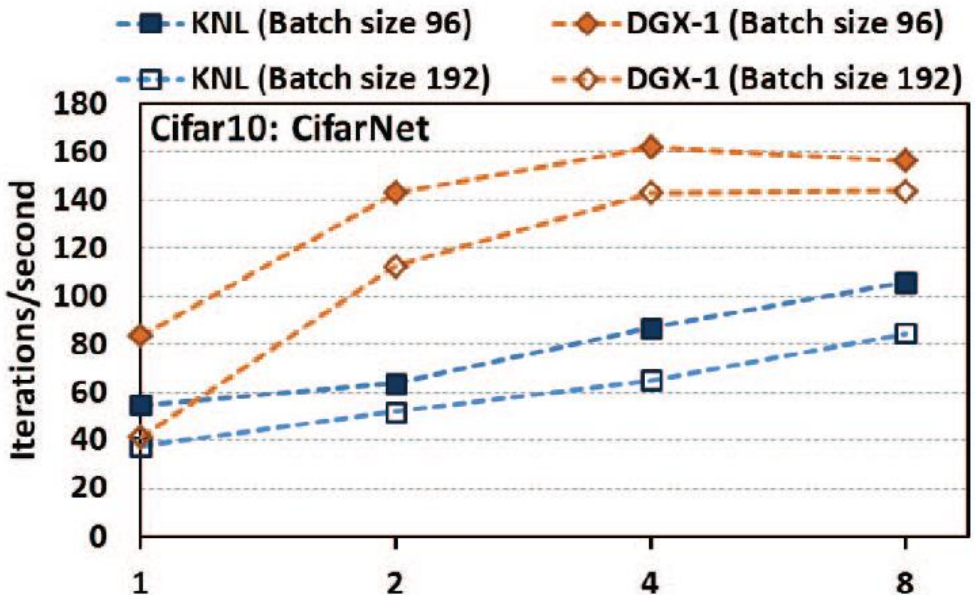
Optimizing KNL

- For single-node execution on KNL, they evaluate three configurations:
- (c1) quadrant clustering with MCDRAM in cache mode
- (c2) quadrant clustering with MCDRAM in flat mode
- (c3) all-to-all clustering with MCDRAM in flat mode.
- In decreasing order of performance, these configurations are (c1), (c2) and (c3).
- Hence, they use configuration (c1) for scaling experiments with KNL.
- They evaluate 1, 2, 4 and 8 GPUs on DGX-1 and 1, 2, 4 and 8 KNL nodes

© Sparsh Mittal

Results on CifarNet

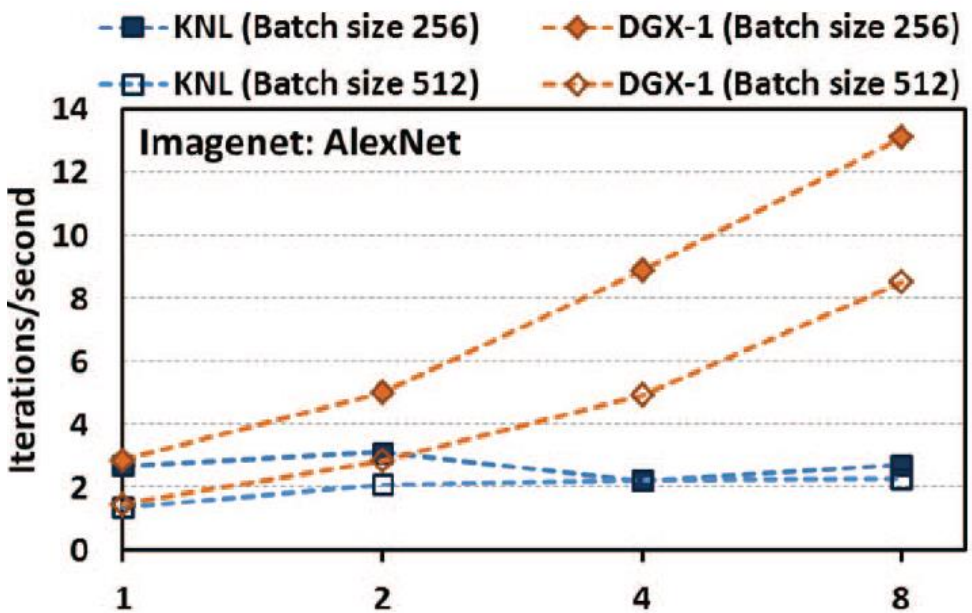
- KNL performance shows sub-linear increase with rising number of nodes due to communication costs
- For DGX-1, performance improves for 2 GPUs due to efficient use of memory of 2 GPUs as buffer during computation and due to high-bandwidth of NVLink
- Performance drops with 4 and 8 GPUs, since compute needs of CifarNet are not large enough to utilize all GPUs



© Sparsh Mittal

Results on AlexNet

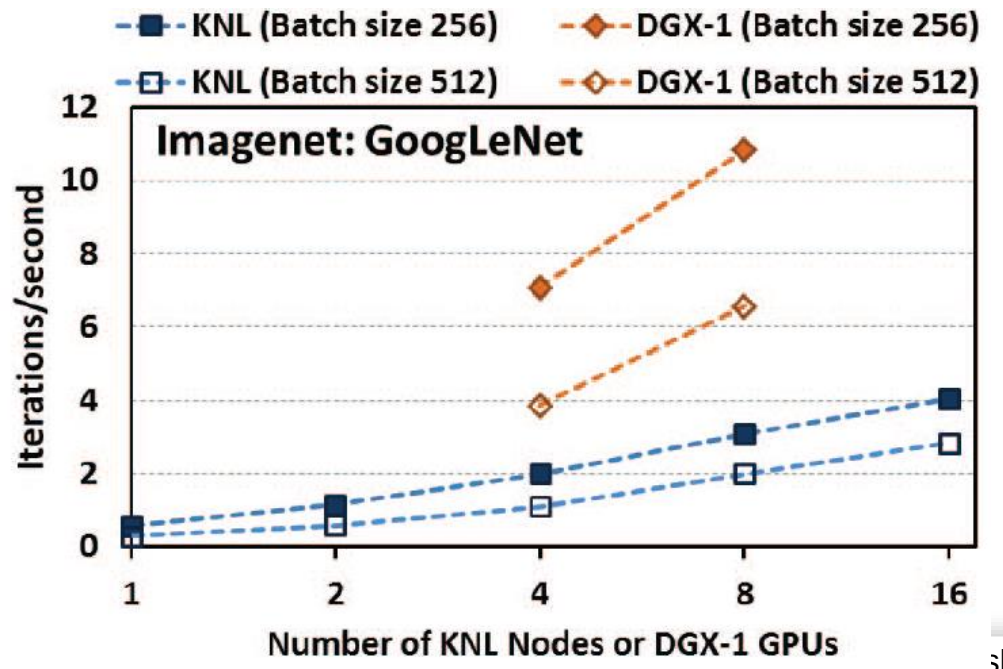
- DGX-1 and KNL give similar results for one node/GPU
- But performance of KNL does not scale with nodes whereas that of DGX-1 scales well, although not linearly



© Sparsh Mittal

Results on GoogleNet

- GoogleNet could not run on 1 or 2 GPUs due to its high memory requirement
- DGX-1 with 8 GPUs shows higher performance than even KNL with 16 nodes.
- Performance of KNL improves slowly with rising number of nodes



Conclusion from Gawande et al' paper

- GPU provides higher raw performance and NVLink is crucial for GPU scaling.
- KNL can provide performance/watt comparable to that of GPU.

© Sparsh Mittal

Deep Learning on KNM

Georganas et al., “Anatomy Of High-Performance Deep Learning Convolutions On SIMD Architectures”, SC, 2018. Author affiliation: Intel Corporation.

© Sparsh Mittal

Implementing CONV

- Observation: implementing CONV as matrix-multiplication leads to large memory footprint and bandwidth dependency.
- Idea: Implement CONV using direct CONV which avoids expensive memory accesses due to shuffle/scatter/gather operations.
- For direct CONV, use of static compilation provides near-peak performance for a given architecture …
- …But incurs overhead of static compilation and requires individual fine-tuning for every CNN topology
- Especially for small matrices, static compilation is not optimal on x86

© Sparsh Mittal

Use of dynamic compilation

- Idea: Since the matrices used in CNNs are generally small, they use dynamic compilation approach for generating microkernels for fast direct convolutions.
- This just-in-time (JIT) compilation approach also avoids recompilation and tuning overheads.

© Sparsh Mittal

Vectorization and Register blocking

- Further, they vectorize computation of output feature maps, since they can be computed independently
- For AVX512 and FP32 datatype, VLEN is 16
- Algorithm 2 shows convolution loops rewritten in a way that exposes register blocking and vectorization opportunities.
- Register blocking factors are RB_P and RB_Q

© Sparsh Mittal

Register blocking

- Register blocking is performed in spatial domain of output tensor since points in the spatial iteration space can be computed in parallel

Input tensor dimensions: N, C, H and W
Output tensor dimensions: N, K (output feature maps), P and Q (output spatial dimensions).

Algorithm 1 Naive forward propagation loops

```
for n = 0 ... N − 1 do
 for k = 0 ... K − 1 do
  for c = 0 ... C − 1 do
   for oj = 0 ... P − 1 do
    for oi = 0 ... Q − 1 do
     ij = stride * oj
     ii = stride * oi
     for r = 0 ... R − 1 do
      for s = 0 ... S − 1 do
       O[n][k][oj][oi] += I[n][c][ij + r][ii + s] * W[k][c][r][s]
```

Algorithm 2 Forward propagation with register blocking

```
C_b = C/VLEN
K_b = K/VLEN
P_b = P/RB_P
Q_b = Q/RB_Q
for n = 0 ... N − 1 do
 for k_b = 0 ... K_b − 1 do
  for c_b = 0 ... C_b − 1 do
   for ojb = 0 ... P_b − 1 do
    for oib = 0 ... Q_b − 1 do
     ij = stride * ojb * RB_P
     ii = stride * oib * RB_Q
     oj = ojb * RB_P
     oi = oib * RB_Q
     for r = 0 ... R − 1 do
      for s = 0 ... S − 1 do
       for k = 0 ... VLEN do
        for c = 0 ... VLEN do
         for p = 0 ... RB_P do
          for q = 0 ... RB_Q do
           ij' = ij + stride * p
           ii' = ii + stride * q
           O[n][k_b][oj+p][oi+q][k] += W[k_b][c_b][r][s][c][k] *
                                       I[n][c_b][ij' + r][ii' + s][c]
```

© Sparsh Mittal

Cache blocking, prefetching and multithreading

- To maximize data reuse from cache, they also apply cache blocking in feature map and spatial dimensions
- Prefetching is used to bring tensor-data used by same and future microkernels.
- Parallelize microkernel execution to available threads

Algorithm 3 Forward propagation with microkernel calls

```
for n = 0 ... N − 1 do
 for k_b = 0 ... K_b − 1 do
  for c_b = 0 ... C_b − 1 do
   for ojb = 0 ... P_b − 1 do
    for oib = 0 ... Q_b − 1 do
     ij = stride * ojb * RB_P
     ii = stride * oib * RB_Q
     oj = ojb * RB_P
     oi = oib * RB_Q
     CONV(&I[n][c_b][ij][ii][0],&W[k_b][c_b][0][0][0][0],&O[n][k_b][oj][oi][0])
```

© Sparsh Mittal

Layer fusion

- Layers such as Activation, Pooling, Normalization are bandwidth-bound (low arithmetic intensity)
- Fusion means these layers are applied when the data-item is present in cache, which saves bandwidth
- It introduces conditional statements for deciding when the operator can be applied.

Algorithm 4 Forward propagation with fused layer L()

```
for n = 0 ... N − 1 do
 for k_b = 0 ... K_b − 1 do
  for c_b = 0 ... C_b − 1 do
   for ojb = 0 ... P_b − 1 do
    for oib = 0 ... Q_b − 1 do
     ij = stride * ojb * RB_P
     ii = stride * oib * RB_Q
     oj = ojb * RB_P
     oi = oib * RB_Q
     CONV(&I[n][c_b][ij][ii][0],&W[k_b][c_b][0][0][0][0],&O[n][k_b][oj][oi][0])
     if fuse(L()) AND c_b == C_b − 1 then
      APPLY(L(), &O[n][k_b][oj][oi][0]))
```

© Sparsh Mittal

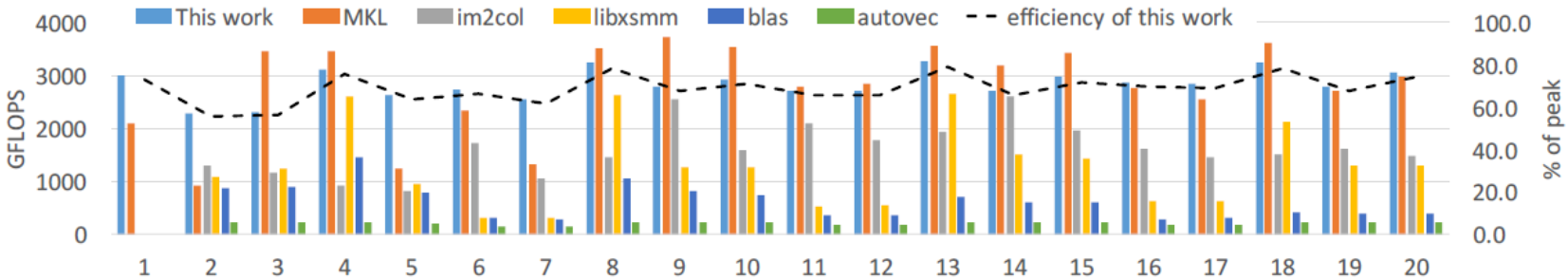
Reduced precision

- 4VNNIW instruction is used which takes int16 inputs and multiplies and accumulates into int32 values
- For ResNet50, KNM offers 2x throughput over FP32 using 4VNNIW.
- **Experiments:** For framework, they use Graph execution Model (GxM), which is a light-weight sibling of Tensorflow
- Results are presented for
  - (1) forward/backward/weight-update propagation kernels and
  - (2) full network execution

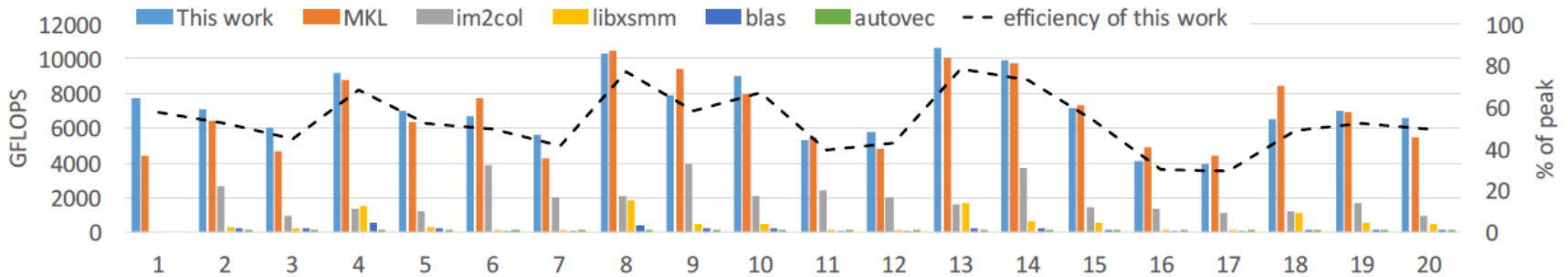
© Sparsh Mittal

Forward propagation results on ResNet50

Skylake CPU



KNM



- KNM performance > Skylake CPU performance
- But % of peak perf. achieved by Phi is lower than that of CPU
- Reason: Phi lacks a shared LLC that absorbs majority of data-movement operations

© Sparsh Mittal

Benefit of reduced precision

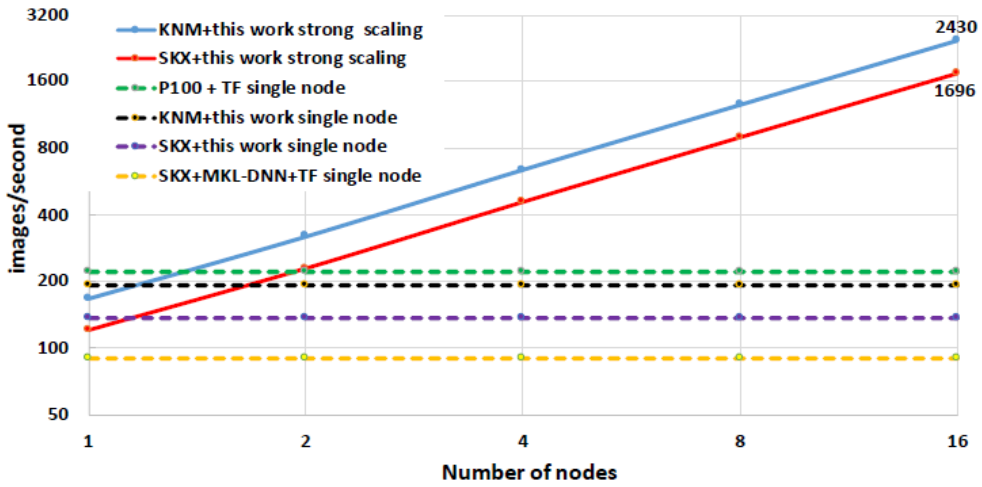
- On Phi, INT16 kernels provide 1.63X speedup over FP32 kernels.
- Speedup is <2X since even though tensor size is half, output is still 32b and hence, its bandwidth requirements are not reduced.

- Skylake CPU doesn't support INT16, so experiments are not performed on CPU.

© Sparsh Mittal

Results on full-network with single node

- They compare performance of training ResNet-50
  - using GxM on CPU/Phi
  - using Tensorflow & cuDNN on NVIDIA P100 GPU
- Throughput on CPU, Phi and GPU are 136, 192 and 219 image/second.



© Sparsh Mittal

Results on full-network with 16 nodes

- “Distributed deep-learning”: strong-scaling to 16 nodes which uses 896 CPU core and 1152 Phi cores
- This increases performance of CPU and Phi to 1696 and 2430 image/second, respectively.
- Tensorflow is responsible for large overhead, due to inefficient thread scheduling, lack of layer fusion, etc.

© Sparsh Mittal

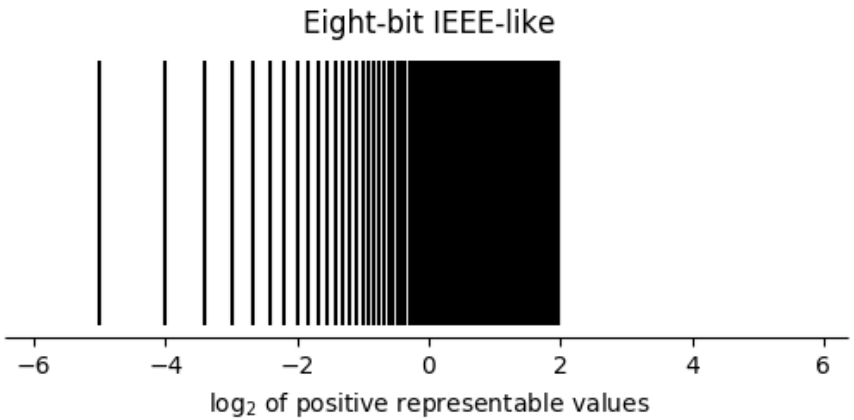
Mixed-precision training of CNNs using integer operations on KNM Phi

Das et al., "Mixed Precision Training of Convolutional Neural Networks using Integer Operations", ICLR 2018

© Sparsh Mittal

Mixed-precision training

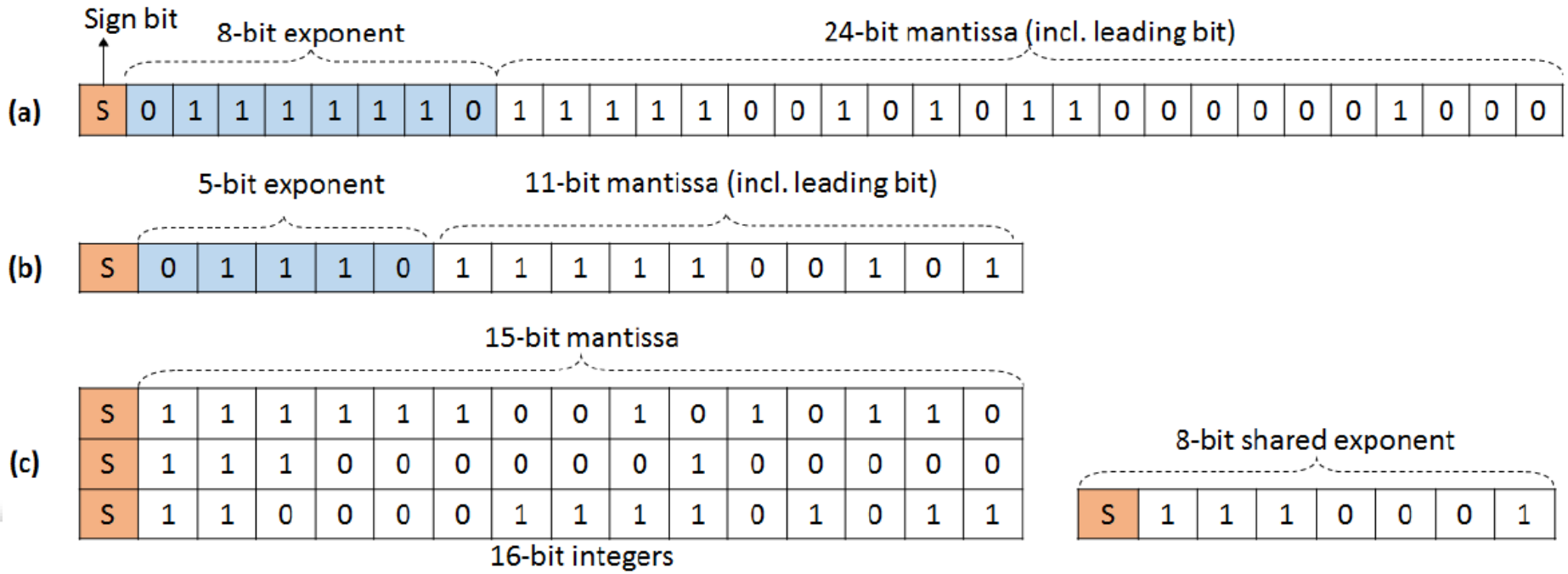
- Keep precision-sensitive operations such as stochastic gradient descent in single-precision
- and computation-intensive operations in half-precision.
- Half-precision training may use either FP16 or INT16.
- Compared to FP16, INT16 provides lower dynamic range but higher precision.



© Sparsh Mittal

Dynamic fixed point (DFP)

- They present DFP format which represents tensors using a combination of an integer tensor I and an exponent Es, shared between all integer elements.
- DFP tensors are shown as DFP-Q where Q is bit-width of integer elements, e.g., DFP-16 has 16b integers.



Benefits of DFP

- DFP-16 has higher computation density than FP32
- and higher effective precision than FP16
  - Since it uses 15b mantissa vs. 11b mantissa in FP16.
- They define
  - multiply/add operations on DFP tensors and
  - conversion-operations between DFP and float.
  - Techniques for handling overflow when results on INT16 operands are stored in INT32 storage

© Sparsh Mittal

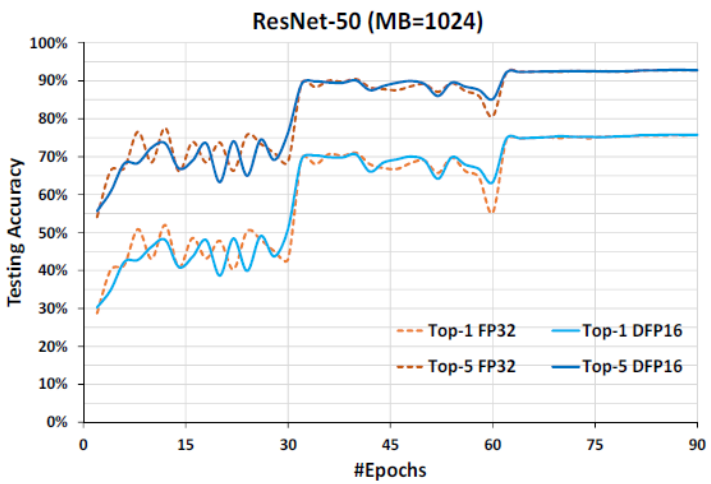
- Since forward/back-propagation and weight-gradient computations can be modeled as GEMM-like convolution operations, they implement these kernels using INT16 operations,
  - E.g., using AVX512_4VNNI instruction.
- These kernels are interfaced with other CNN training operations through DFP to floating-point conversions.
- Storing the result of FMA operation between INT16 operands into 32b operand can lead to overflow
- Hence, they discuss technique for overflow management.

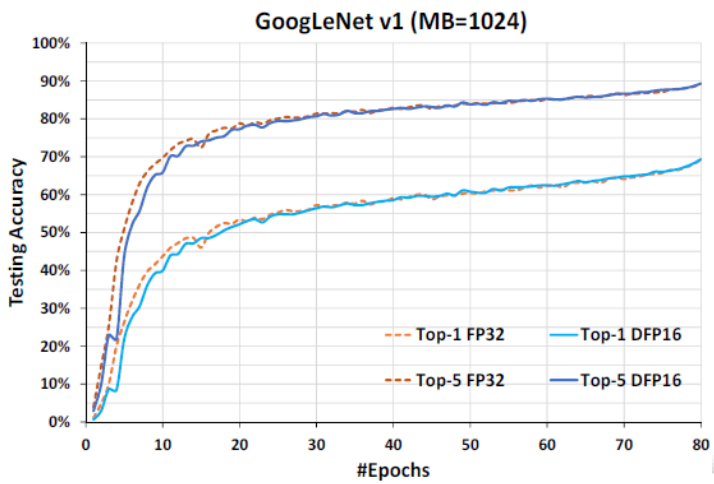
© Sparsh Mittal

Results

- Top-1 accuracies achieved by their technique equals or exceeds that achieved by FP32 training.
- This is achieved in the same number of iterations as FP32 training and with the same hyper-parameters.

Models	Batch-size / Epochs	Baseline		Mixed precision DFP16	
		Top-1	Top-5	Top-1	Top-5
ResNet-50	1024 / 90	75.70%	92.78%	75.77%	92.84%
GoogLeNet-v1	1024/ 80	69.26%	89.31%	69.34%	89.31%
VGG-16	256 / 60	68.23%	88.47%	68.12%	88.18%
AlexNet	1024 / 88	57.43%	80.65%	56.94%	80.06%

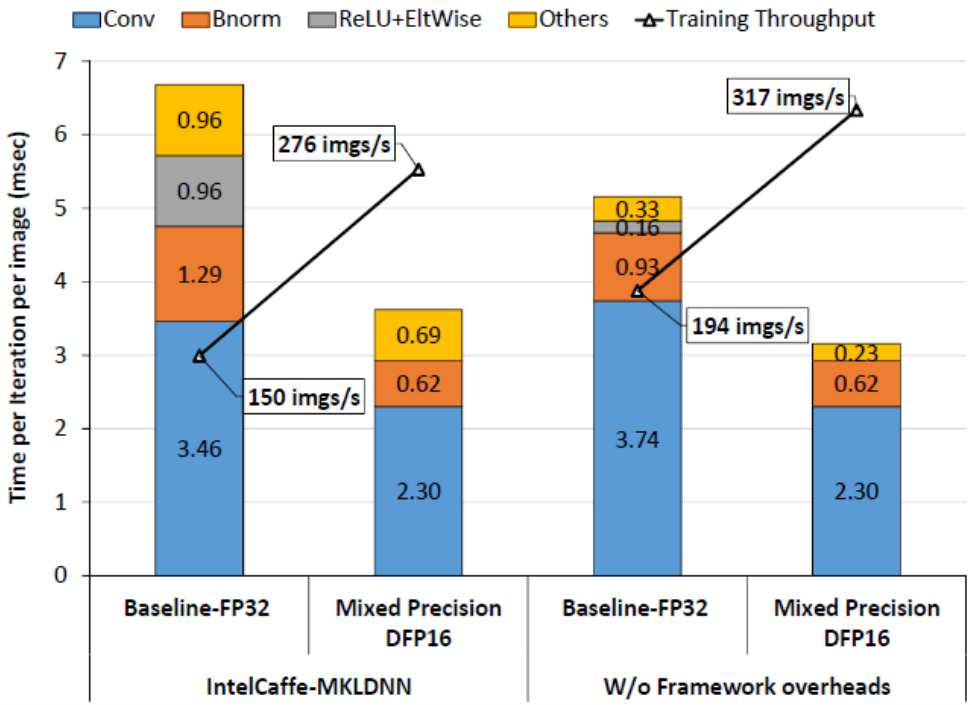




© Sparsh Mittal

Their techniques also improves throughput

- 3x3 kernels are 1.8X faster and 1x1 kernels are 1.4X faster
- Bnorm benefits due to double bandwidth
- ReLU+EltWise are fused with batchnorm



© Sparsh Mittal

(Intel's) Turbo Boost Technology

- Enable temporary higher performance by increasing the processor's clock frequency above the maximum safe frequency of a processor.
- frequency is increased algorithmically when when there is sufficient headroom - subject to power rating, temperature rating, and current limits.

- Turbo frequency can be used for some number of milliseconds until the thermal credits are exhausted
- At this point, processor drops into a slower, cooler CPU clock frequency to adhere to thermal limits.

© Sparsh Mittal

(Intel's) Turbo Boost Technology

- With only one core running, turbo frequency can be much higher than with multiple cores running.

- Ex: Sandy Bridge-based quad-core i7-2960XM has a base frequency of 2.7 GHz. Turbo-boost max frequency with
- 1 core: 3.7GHz.
- 2 core: 3.6GHz
- 3 or 4 core: 3.5GHz

© Sparsh Mittal

Impact of low-precision on HPC applications

Domke et al, “Double-precision FPUs in High-Performance Computing: an Embarrassment of Riches?”, Arxiv 2018

© Sparsh Mittal

HPL and HPCG (L=LINPACK, CG=Conjugate Gradient)

- HPL solves a dense system of linear equations. Very few such applications exist.
- HPL benchmark is very regular: FLOP= N^3, Bytes = N^2
- Performance achieved on HPL is quite high and reflects peak performance attainable on a system.
- HPCG uses sparse data structures. FLOP =N, Bytes = N. => Low arithmetic intensity
- HPCG performance: strongly influenced by memory bandwidth.
- HPCG models a large class of applications. HPCG performance is a lower bound on the achievable performance for many applications.

© Sparsh Mittal

Goal

- **Observation:** traditional processors allocated a large portion of silicon area to DP computing units
- Recent processors, e.g. KNM, allocate a large portion of chip area to single/half-precision/integer units.
- **Goal:** To study the impact of this change on the performance of HPC applications.
- Benchmarks: Total 22 applications including HPL, HPCG and apps from exascale computing project proxy-apps and RIKEN fiber miniapp suite.

© Sparsh Mittal

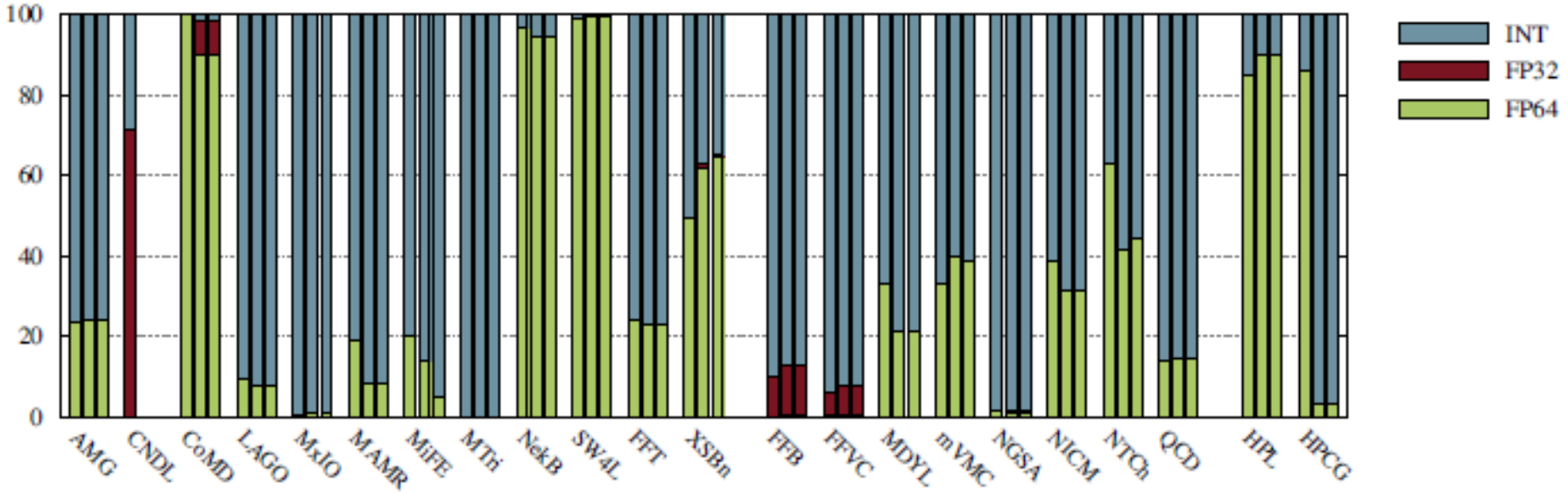
ECP	Scientific/Engineering Domain	Compute Pattern	Language
AMG	Physics and Bioscience	Stencil	C
CANDLE	Bioscience	Dense matrix	Python
CoMD	Material Science/Engineering	N-body	C
Laghos	Physics	Irregular	C++
miniAMR	Geoscience/Earthscience	Stencil	C
miniFE	Physics	Irregular	C++
miniTRI	Math/Computer Science	Irregular	C++
Nekbone	Math/Computer Science	Sparse matrix	Fortan
SW4lite	Geoscience/Earthscience	Stencil	C
SWFFT	Physics	FFT	C/Fortran
XSBench	Physics	Irregular	C

RIKEN	Scientific/Engineering Domain	Compute Pattern	Language
FFB	Engineering (Mechanics, CFD)	Stencil	Fortran
FFVC	Engineering (Mechanics, CFD)	Stencil	C++/Fortran
mVMC	Physics	Dense matrix	C
NICAM	Geoscience/Earthscience	Stencil	Fortran
NGSA	Bioscience	Irregular	C
MODYLAS	Physics and Chemistry	N-body	Fortran
NTChem	Chemistry	Dense matrix	Fortran
QCD	Lattice QCD	Stencil	Fortran/C

© Sparsh Mittal

Ratio of Integer vs. FP32 vs FP64

- Out of 22 apps, only 4 rely on FP32 instructions and only one uses both FP32 and FP64 instructions.
- 16 apps execute at least 50% integer instructions.

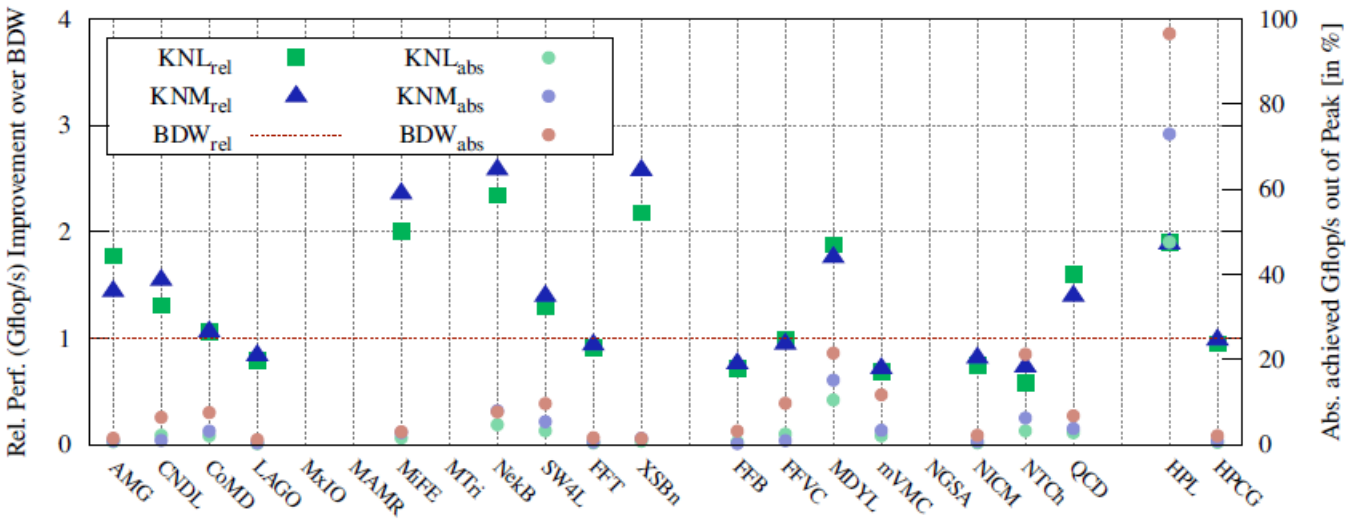


Left bar = Broadwell (BDW), middle bar = KNL, right bar = KNM;

© Sparsh Mittal

Relative (FP32+FP64) of KNL/KNM over Broadwell and Absolute per. over peak perf.

- Except HPL, all apps achieve at most 10.5% FP efficiency (i.e., percentage of peak performance) on KNL, 15.1% FP efficiency on KNM and 21.5% FP efficiency on Broadwell CPU.
- This shows that the utility of FP units is limited.



© Sparsh Mittal

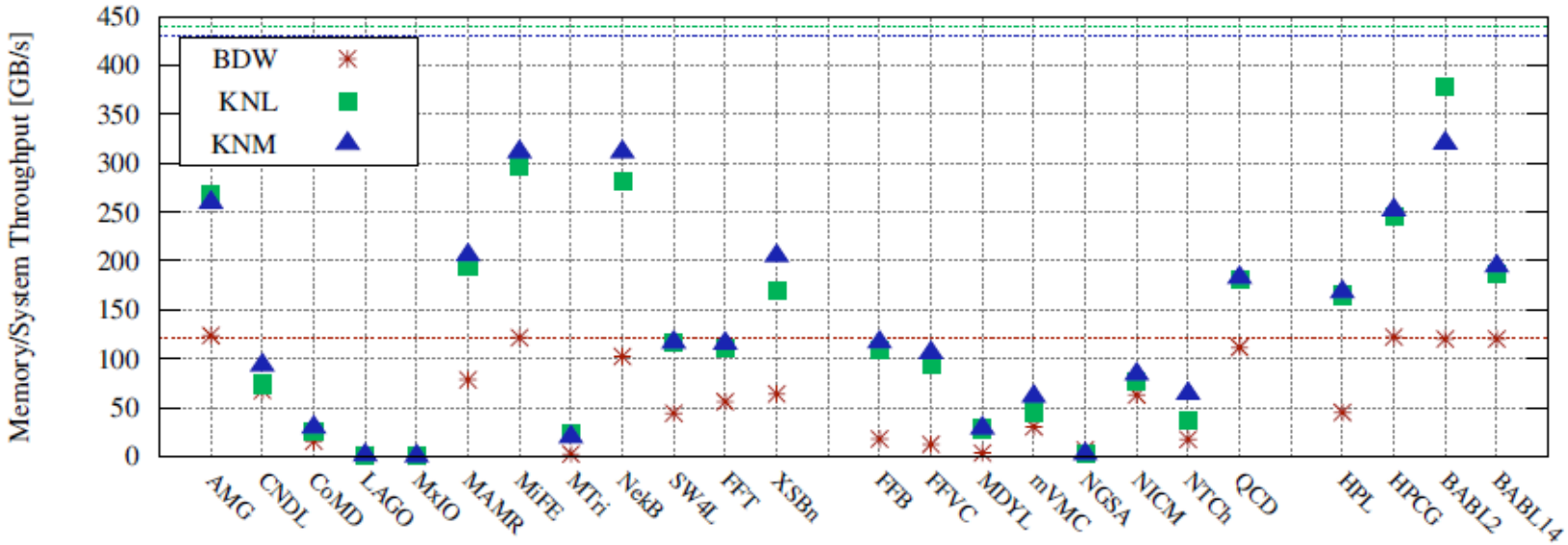
Comparison of KNL vs KNM

- Most apps achieve comparable performance on KNL and KNM.
- Higher perf on KNM:
  - Some apps e.g. CANDLE benefit from VNNI units in mixed precision
  - MiFE, NekB, and XSBn improve due to increased core count and KNM's higher CPU frequency
- Higher perf on KNL
  - Some memory-bound applications (i.e., AMG, HPCG, and MTri) are slower on KNM due to the increased core count causing higher competition for bandwidth.

© Sparsh Mittal

Memory throughput results (1 of 3)

- Memory throughput of KNL and KNM are similar for most apps
- and is higher than that of Broadwell CPU due to presence of MCDRAM on KNL/KNM.



© Sparsh Mittal

Memory throughput results (2 of 3)

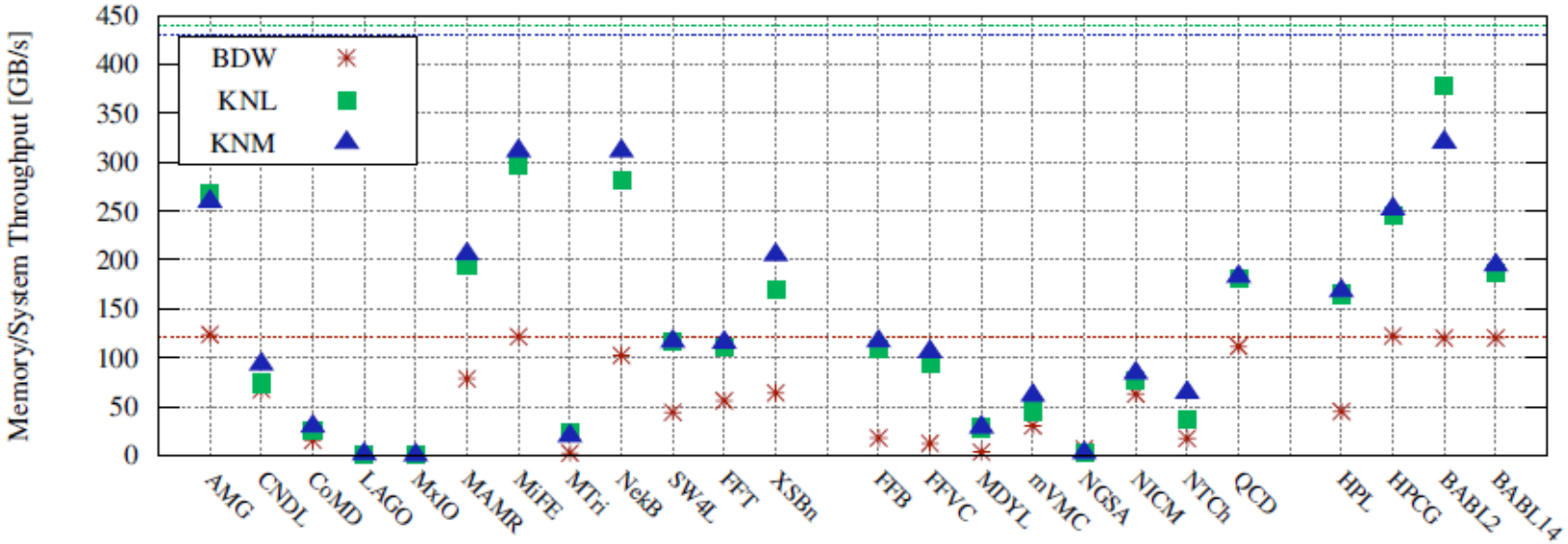
- Throughput advantage of the MCDRAM translates into a performance boost for six proxy-apps (AMG, MAMR, MiFE, NekB, XSBn, and QCD) which heavily utilize available bandwidth and are memory-bound

- **BabelStream:** a benchmark used to measure the memory transfer rates to/from the memory. Similar to the STREAM benchmark

© Sparsh Mittal

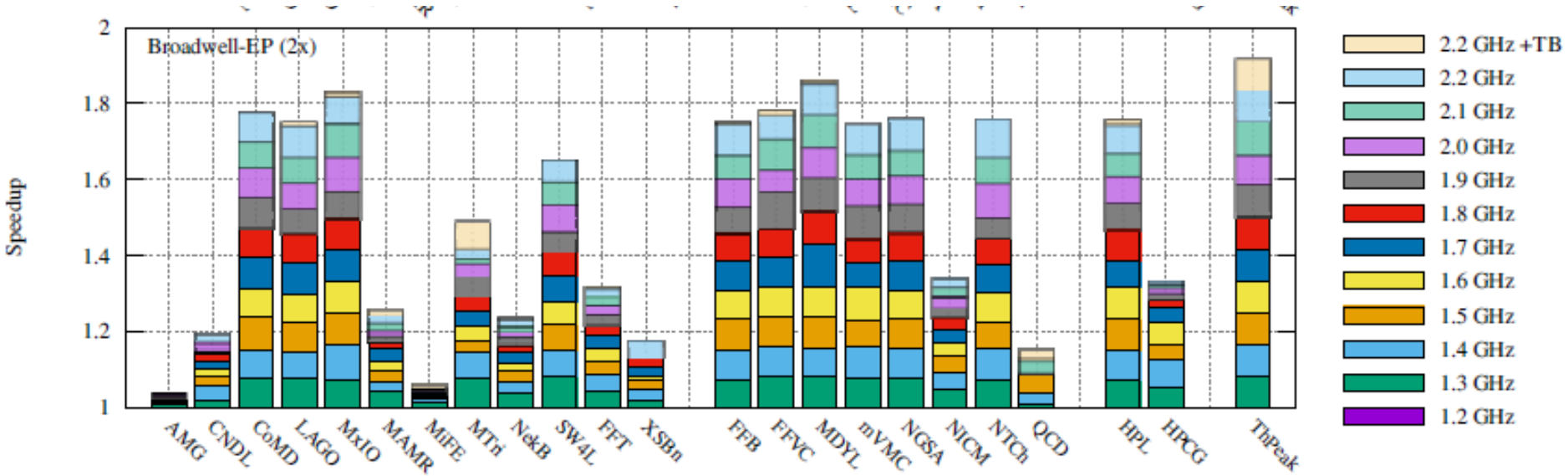
Memory throughput results (3 of 3)

- BABL2: BabelStream with 2GB workload.
  - Achieves 86% on KNL and 75% on KNM since the vectors fit into MCDRAM
- BABL14: BabelStream with 14GB workload.
  - Bandwidth is slightly higher than DRAM throughput (due to minor prefetching benefits) since the vectors do not fit in MCDRAM



Frequency Scaling to Identify Compute-Boundedness

- On Broadwell, only one app (MTri) benefits from Turbo boost.

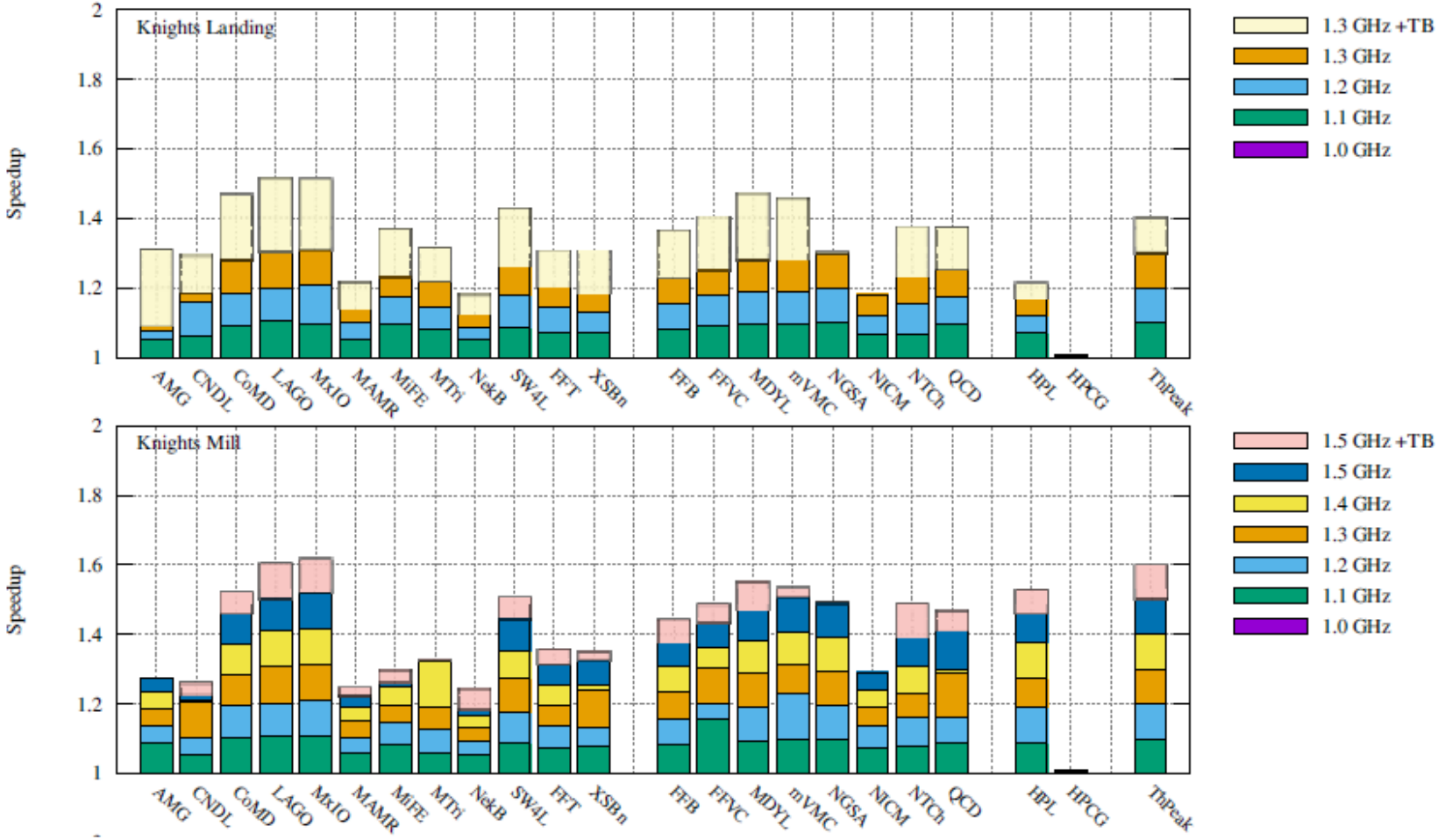


Speedup obtained through increased CPU frequency (w.r.t baseline frequency of 1.0GHz on KNL/KNM and 1.2GHz on BDW);

© Sparsh Mittal

Frequency Scaling to Identify Compute-Boundedness

- On KNL/KNM, nearly-all apps benefit from turbo-boost



© Sparsh Mittal

Frequency Scaling to Identify Compute-Boundedness

- Overall, benchmarks are more compute-bound than memory-bound.
- Memory-bound apps such as AMG and MiniFE benefit very little from increasing core frequency on Broadwell. They benefit from high capacity/bandwidth MCDRAM on KNL and KNM
- HPCG is memory-latency bound and hence, it is not affected by frequency scaling on KNL/KNM.
- Conclusion: reduction in double-precision compute-capability has small or no impact on performance of most apps. This supports the decision to use low/hybrid-precision compute units.

© Sparsh Mittal